

Estruturas em Árvore Especializadas: Modelagem, Implementação e Análise Comparativa de Trie, Patricia, Splay, Treap e KD-Tree

Maria Eduarda Alves Barbosa

Resumo. Este artigo apresenta a modelagem, a implementação e a análise comparativa de cinco estruturas de dados em árvore especializadas: Trie, Árvore Patricia (radix compacta), Árvore Splay, Árvore Treap e KD-Tree, todas implementadas em C++17 com as operações de inserção, busca e remoção, além das operações específicas de cada estrutura. Para cada estrutura são discutidos os fundamentos, os invariantes e as decisões de projeto; apresenta-se o rastreamento visual de três estados representativos; deriva-se a complexidade assintótica (casos melhor, médio e pior); e realizam-se experimentos computacionais que confrontam o comportamento teórico com o observado na prática. Os resultados evidenciam que não existe uma estrutura universalmente superior: cada uma domina um contexto específico: strings e prefixos (Trie/Patricia), acessos com localidade temporal (Splay), balanceamento probabilístico simples (Treap) e busca espacial multidimensional (KD-Tree). Destaca-se ainda uma divergência instrutiva entre teoria e prática no caso da Splay, cuja vantagem sob localidade é parcialmente mascarada por fatores constantes.

Termos de indexação. Estruturas de dados, árvores de busca, Trie, Patricia, Splay, Treap, KD-Tree, análise de complexidade, análise experimental, C++.

I. INTRODUÇÃO

A **organização hierárquica de dados** por meio de árvores é um dos pilares da Ciência da Computação: ao manter a altura próxima do logaritmo do número de elementos, uma árvore reduz o custo de busca de $O(n)$ para $O(\log n)$. As Árvores Binárias de Busca (BST) exploram essa ideia, mas degeneram para uma lista encadeada, com altura $O(n)$, quando as chaves são inseridas em ordem desfavorável. As Árvores AVL corrigem o problema com balanceamento estrito por altura, ao custo de manter informação extra em cada nó e de executar rotações a cada modificação. Contudo, tanto a BST quanto a AVL pressupõem chaves escalares comparáveis e acessos igualmente prováveis, hipóteses que muitos problemas reais não satisfazem.

A. Contexto histórico

As estruturas estudadas neste trabalho acompanham a própria evolução da área. A **Trie** foi proposta por Fredkin em 1960 como mecanismo de recuperação de

informação por prefixos; a **Árvore Patricia** foi formalizada por Morrison em 1968 [6] para indexação compacta de dados alfanuméricos. Na linha das árvores de busca autoajustáveis, Sleator e Tarjan introduziram a **Splay** em 1985 [3], que abandona o balanceamento explícito em favor de um equilíbrio amortizado guiado pelo padrão de acesso. Aragon e Seidel propuseram a **Treap** em 1989 [4], unindo a busca binária a prioridades aleatórias para obter balanceamento probabilístico com implementação simples. Para dados espaciais, Bentley apresentou a **KD-Tree** em 1975 [5], generalizando a busca binária a múltiplas dimensões. Todas surgiram para superar limitações das BST e das AVL (Adelson-Velsky e Landis, 1962) em cenários específicos.

B. Motivação e objetivos

Muitos problemas fogem das hipóteses de BST/AVL: buscas por **prefixos de strings**, padrões de acesso com **localidade temporal** (poucas chaves acessadas com muita frequência), necessidade de uma estrutura **balanceada em média com implementação simples**, ou consultas em **espaços multidimensionais**. Cada uma das cinco estruturas adota uma estratégia de organização distinta para atender a essas necessidades. Este trabalho tem quatro objetivos: (a) fundamentar teoricamente as cinco estruturas e seus invariantes; (b) implementá-las em C++17; (c) demonstrar visualmente sua evolução ao longo das operações; e (d) avaliar empiricamente seu desempenho, discutindo convergências e divergências em relação à análise assintótica. O texto está organizado da fundamentação teórica (Seção II) à conclusão (Seção VIII), privilegiando, conforme recomendado, a análise crítica e comparativa em vez da descrição isolada de cada estrutura.

II. FUNDAMENTAÇÃO TEÓRICA

A. Conceitos fundamentais das estruturas

1) *Trie (árvore de prefixos)*: árvore n -ária em que cada aresta representa um caractere; uma chave corresponde ao caminho da raiz até um nó marcado como fim de palavra, de modo que prefixos comuns compartilham o mesmo caminho. **Invariante**: o caminho da raiz a um nó descreve exatamente o prefixo acumulado. **Anatomia do nó**: um mapa de filhos indexado por caractere e um marcador booleano de fim de palavra. Resolve, naturalmente, a busca por prefixos e o armazenamento de conjuntos de strings.

2) *Árvore Patricia (radix compacta)*: Trie comprimida em que cadeias de nós com um único filho são colapsadas, de forma que cada aresta guarda uma **substring** em vez de um único caractere. **Invariante**: nenhum nó interno (exceto a raiz) possui um único filho sem também ser fim de chave. As operações empregam **divisão (split)** de arestas na inserção e **fusão (merge)** na remoção, reduzindo drasticamente o número de nós frente à Trie e, portanto, o consumo de memória.

3) *Árvore Splay*: BST autoajustável: a cada acesso, o nó tocado é levado à raiz pela operação **splay** (casos zig, zig-zig e zig-zag). Não armazena altura nem fator de balanceamento (o equilíbrio é apenas **amortizado**) e elementos acessados recentemente ficam próximos da raiz, favorecendo padrões com localidade temporal de referência.

4) *Árvore Treap*: combina uma BST pelas **chaves** com um max-heap pelas **prioridades**, sorteadas aleatoriamente na inserção. Como as prioridades são aleatórias, a forma da árvore equivale à de uma BST construída em ordem aleatória, resultando em altura esperada $O(\log n)$ sem regras de balanceamento complexas; o equilíbrio entre as duas propriedades é mantido por rotações simples.

5) *KD-Tree (k -dimensional, $k = 2$)*: organiza pontos alternando a dimensão de corte a cada nível (nível par corta em x ; nível ímpar, em y). O plano é sucessivamente particionado em retângulos, o que viabiliza consultas espaciais eficientes, como a busca do vizinho mais próximo, aproveitando a poda de regiões que não podem conter a resposta.

B. Diferenças conceituais em relação a BST e AVL

- **Tipo de chave**: BST/AVL/Splay/Treap comparam chaves escalares; Trie/Patricia operam sobre a estrutura interna das strings; a KD-Tree compara coordenadas por dimensão.
- **Balanceamento**: a AVL usa balanceamento estrito por altura; a Splay, reorganização por acesso (amortizado); a Treap, aleatoriedade (probabilístico); Trie/Patricia dispensam balanceamento, pois a altura depende do comprimento das chaves, não de n .
- **Custo em função de n** : em Trie/Patricia as operações são $O(m)$ e independem de n ; nas demais, dependem da altura da árvore, cujo controle é justamente o que diferencia cada estratégia.

III. PROJETO E IMPLEMENTAÇÃO DAS ESTRUTURAS

As cinco estruturas foram implementadas em C++17, cada uma em um cabeçalho independente (header-only) na pasta `include/`. As entradas pseudoaleatórias usam semente fixa (42), garantindo **reprodutibilidade**. Descrevem-se a seguir as principais decisões de projeto; o código-fonte completo está no repositório indicado nas referências, evitando-se a reprodução integral do código no corpo do artigo.

A. Trie e Patricia

Na **Trie**, cada nó mantém um `std::map<char, Node*>` (mapa ordenado, para saída alfabética) e um booleano `is_end`. A inserção percorre a chave criando nós ausentes; a busca segue o caminho e verifica o marcador final; a remoção desmarca o fim de palavra e, ao retornar da recursão, apaga nós que ficaram sem filhos e sem marcação. Armazenar o marcador de fim no próprio nó permite que uma palavra seja prefixo de outra sem ambiguidade.

A **Patricia** acrescenta a cada nó o rótulo da aresta que vem do pai. A inserção calcula o maior prefixo comum entre a chave e o rótulo corrente; quando o prefixo é apenas parcial, a aresta é **dividida**, criando um nó intermediário (**split**). A remoção pode **fundir** um nó que ficou com um único filho ao seu descendente (**merge**), concatenando rótulos e preservando o invariante de compactação, a principal diferença de projeto frente à Trie. O Algoritmo 1 resume a inserção com split.

Algoritmo 1. Inserção com split na Patricia

```

1: função INSERIR(nó, chave)
2:  L ← rótulo da aresta até nó
3:  k ← comprimento do maior prefixo comum(chave, L)
4:  se k < |L| então      > split
5:    quebra a aresta em k; cria nó intermediário
6:  se k = |chave| então marca fim de chave
7:  senão INSERIR(filho[chave[k]], sufixo(chave, k))

```

B. Splay e Treap

A **Splay** usa ponteiros de pai para viabilizar o splay ascendente (Algoritmo 2). Inserção e busca são operações de BST seguidas do splay do nó acessado; a remoção dá splay na chave, remove a raiz e **junta** as subárvores (leva o máximo da esquerda à raiz e anexa a direita). Os três casos de rotação (zig, zig-zig e zig-zag) estão implementados explicitamente, pois deles depende a garantia amortizada.

Algoritmo 2. Operação splay (subida do nó x)

```

1: função SPLAY(x)
2:  enquanto pai(x) ≠ nulo faça
3:    se avô(x) = nulo então ZIG(x)
4:    senão se x e pai(x) do mesmo lado então
5:      ZIG-ZIG(x)
6:    senão ZIG-ZAG(x)

```

A **Treap** é recursiva: na inserção, após descer ao filho, se a prioridade do filho superar a do pai aplica-se uma rotação que o faz subir, restaurando o max-heap. Na remoção, o nó é rotacionado para baixo (subindo sempre o filho de maior prioridade) até virar folha, quando é eliminado sem afetar o restante da árvore. As prioridades são geradas por um `std::mt19937` de semente fixa, o que torna os experimentos reproduzíveis.

C. KD-Tree

Cada nó guarda um ponto (x, y) e a dimensão de corte é dada pela profundidade (`depth % 2`). Inserção e busca exata comparam apenas a coordenada do nível. A remoção emprega a técnica

clássica: um nó interno é substituído pelo **mínimo, na dimensão de corte, da subárvore direita** (`findMin`); sem subárvore direita, usa-se a esquerda, que passa a ser a direita. A busca do **vizinho mais próximo** (Algoritmo 3) desce primeiro para o lado que contém o alvo e só visita o outro se a distância ao plano de corte for menor que a melhor já encontrada (poda), o que sustenta o desempenho médio logarítmico.

Algoritmo 3. Vizinho mais próximo na KD-Tree

```

1: função VMP(nó, alvo, prof, melhor)
2:  se nó = nulo então retorna melhor
3:  atualiza melhor com dist(nó.ponto, alvo)
4:  d ← prof mod 2 ; Δ ← alvo[d] menos nó.ponto[d]
5:  (perto, longe) ← (Δ < 0 ? esq, dir : dir, esq)
6:  melhor ← VMP(perto, alvo, prof+1, melhor)
7:  se Δ² < melhor.dist então
8:    melhor ← VMP(longe, alvo, prof+1, melhor)

```

IV. DEMONSTRAÇÃO E RASTREAMENTO VISUAL

Para cada estrutura exibem-se três estados: (1) inicial, após um conjunto de inserções; (2) intermediário, evidenciando uma operação relevante (bifurcação, split, rotação ou particionamento); e (3) após uma remoção/reorganização. As figuras foram geradas automaticamente a partir da topologia exportada pelas próprias implementações, garantindo fidelidade e rastreabilidade. Nas figuras de Trie/Patricia, nós verdes com * indicam fim de chave, nós cinza são internos e os rótulos em vermelho sobre as arestas são os caracteres/substrings.

A. Trie

Inserção de *casa*, *caso* (prefixo comum *cas*); em seguida *carro* cria uma bifurcação em *ca*; por fim remove-se *caso*. Observa-se o compartilhamento de prefixos característico da Trie.

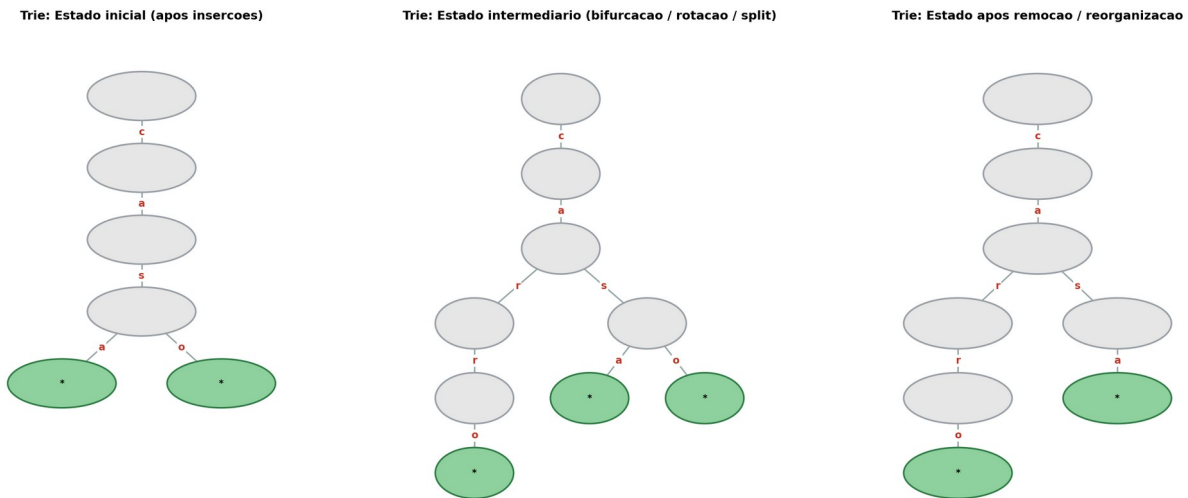


Fig. 1. Trie: estado inicial, bifurcação após inserir “carro” e estado após remover “caso”.

B. Árvore Patricia

Inserção de corda, cordão, corte; depois casa provoca um split do prefixo em c; por fim

remove-se corte. As arestas guardam substrings inteiras, reduzindo o número de nós frente à Trie e evidenciando a compactação.

Patricia (Radix compacta): Estado inicial (apos insercoes)

Patricia (Radix compacta): Estado intermediario (bifurcacao / rotacao / split)

Patricia (Radix compacta): Estado apos remocao / reorganizacao

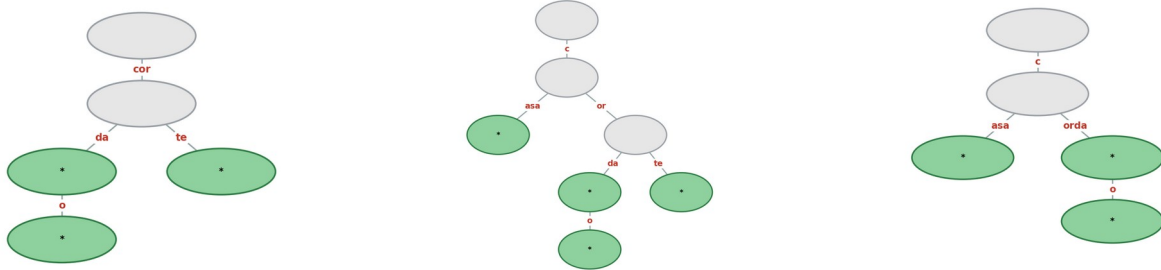


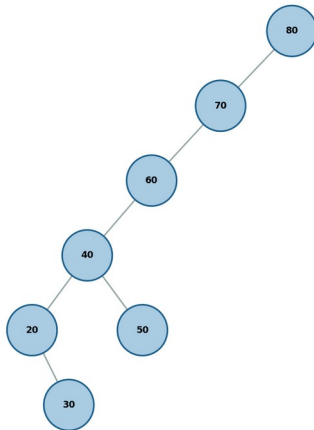
Fig. 2. Patricia: compactação por substrings, split ao inserir “casa” e recompactação após remoção.

C. Árvore Splay

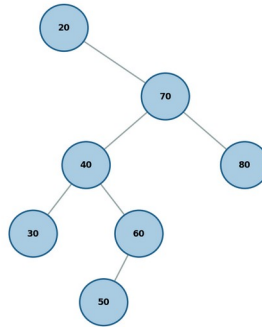
Após inserir 50 30 70 20 40 60 80, uma busca por 20 leva o nó acessado à raiz por rotações (estado

intermediário); em seguida remove-se 20, evidenciando o autoajuste que aproxima da raiz o que foi acessado recentemente.

Arvore Splay: Estado inicial (apos insercoes)



Arvore Splay: Estado intermediario (bifurcacao / rotacao / split)



Arvore Splay: Estado apos remocao / reorganizacao

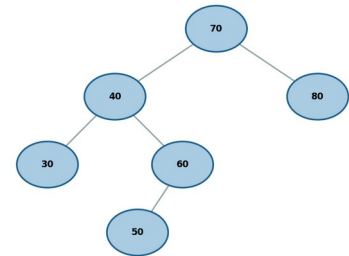


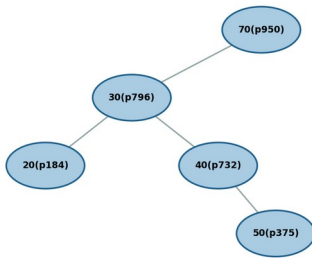
Fig. 3. Splay: estado inicial, splay de 20 até a raiz após a busca e estado após remover 20.

D. Árvore Treap

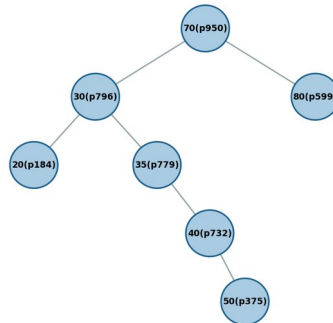
Cada nó mostra chave(prioridade); as chaves respeitam a BST e as prioridades, o max-heap (a

prioridade do pai supera a dos filhos). A inserção de 35 e 80 provoca rotações; depois remove-se 30. Verifica-se que a aleatoriedade das prioridades mantém a árvore aproximadamente balanceada.

Arvore Treap: Estado inicial (apos insercoes)



Arvore Treap: Estado intermediario (bifurcacao / rotacao / split)



Arvore Treap: Estado apos remocao / reorganizacao

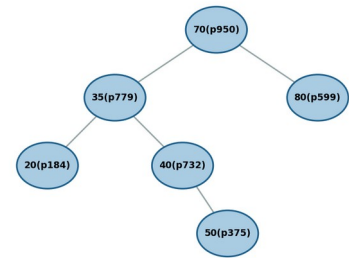


Fig. 4. Treap: BST pelas chaves e max-heap pelas prioridades; rotações na inserção e na remoção.

E. KD-Tree

São inseridos seis pontos; o particionamento do plano mostra os cortes verticais (em x, nível par, em vermelho) e horizontais (em y, nível ímpar, em azul). A inserção de (60,50) subdivide uma região; a

remoção de (30,40) (a raiz) aciona a substituição pelo mínimo (findMin). A busca do vizinho mais próximo de (55,55) retorna corretamente (60,50). A Fig. 6 mostra a topologia correspondente ao estado inicial.

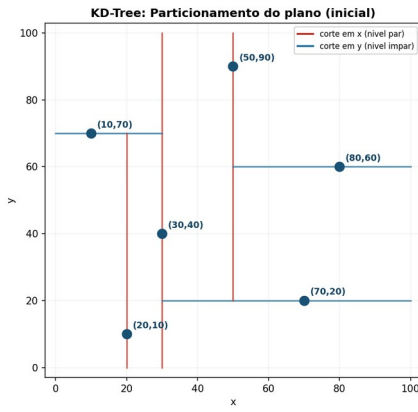


Fig. 5. KD-Tree: particionamento do plano nos três estados (inicial, após inserir (60,50) e após remover (30,40)).

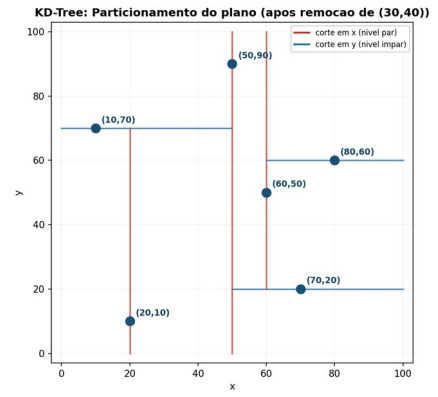
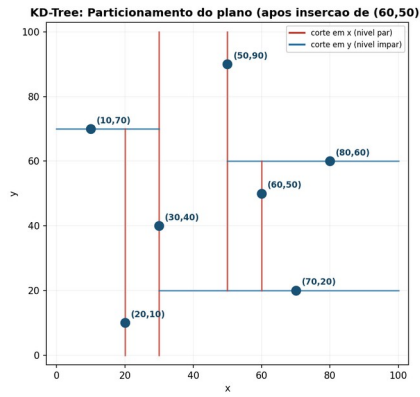
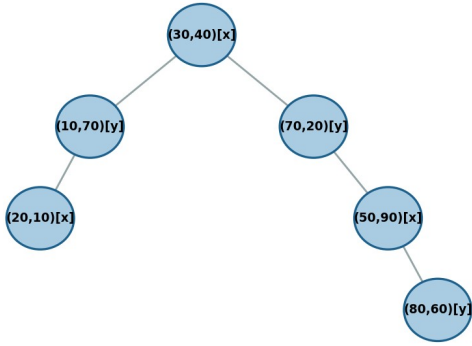


Fig. 6. Topologia da KD-Tree no estado inicial; cada rótulo indica o ponto e a dimensão de corte [x] ou [y].

KD-Tree: Estado inicial (apos insercoes)



V. ANÁLISE DE COMPLEXIDADE E COMPARAÇÃO TEÓRICA

A Tabela I resume a complexidade assintótica das operações. Notação: n = número de elementos; m = comprimento da chave; N = número de chaves; σ = tamanho do alfabeto. Duas expressões separadas por barra indicam **caso médio / pior caso**; ^a = amortizado; ^e = esperado (probabilístico); * = pior caso de espaço da Trie. Não basta reproduzir os valores: a Seção V-A justifica cada custo.

TABELA I
COMPLEXIDADE ASSINTÓTICA DAS OPERAÇÕES (BST E AVL COMO REFERÊNCIA)

Estrutura	Busca	Inserção	Remoção	Espaço	Construção
Trie	$O(m)$	$O(m)$	$O(m)$	$O(N \cdot m \cdot \sigma)^*$	$O(N \cdot m)$
Patricia	$O(m)$	$O(m)$	$O(m)$	$O(N)$	$O(N \cdot m)$
Splay	$O(\log n)^a / O(n)$	$O(\log n)^a$	$O(\log n)^a$	$O(n)$	$O(n \log n)$
Treap	$O(\log n)^e / O(n)$	$O(\log n)^e$	$O(\log n)^e$	$O(n)$	$O(n \log n)$
KD-Tree	$O(\log n) / O(n)$	$O(\log n) / O(n)$	$O(\log n) / O(n)$	$O(n)$	$O(n \log n)$
BST (ref.)	$O(\log n) / O(n)$	$O(\log n) / O(n)$	$O(\log n) / O(n)$	$O(n)$	$O(n \log n)$
AVL (ref.)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n \log n)$

A. Discussão dos custos

Em **Trie** e **Patricia**, as operações custam $O(m)$ por dependerem apenas dos m caracteres da chave, e não de n , uma diferença qualitativa em relação a todas as demais. A Patricia executa menos passos por não visitar nós intermediários redundantes e usa $O(N)$ nós contra o pior caso $O(N \cdot m \cdot \sigma)$ da Trie, o que explica sua vantagem de memória observada empiricamente.

Para a **Splay**, uma operação isolada pode custar $O(n)$ (árvore encadeada), mas o método do potencial,

com $\Phi = \sum_x \log(\text{tam}(x))$, garante que o custo amortizado por operação satisfaz

$$\hat{c}_i = O(\log n), \text{ em qualquer sequência de acessos. (1)}$$

Já a **Treap**, com prioridades aleatórias, comporta-se como uma BST aleatória, cuja profundidade esperada de um nó é

$$E[\text{prof}] \approx 2 \ln n = O(\log n), \quad (2)$$

de modo que o pior caso $O(n)$ existe, mas com probabilidade desprezível, tratando-se de um custo **probabilístico**, não amortizado. A **KD-Tree** tem construção balanceada $O(n \log n)$ e busca $O(\log n)$ em

média; em altas dimensões ou com inserções desfavoráveis, a poda perde eficácia e a busca tende a $O(n)$. Frente a **BST/AVL**, a AVL garante $O(\log n)$ de pior caso, enquanto Splay e Treap abrem mão dessa garantia estrita em troca de simplicidade (Treap) ou de adaptação ao padrão de acesso (Splay).

VI. METODOLOGIA EXPERIMENTAL E RESULTADOS

A. Ambiente e protocolo

Os experimentos estão em `src/bench.cpp`, compilados com `g++ -O2 -std=c++17`. Os tempos, em milissegundos, são medidos com `std::chrono::high_resolution_clock` e somam todas as operações de cada fase. As entradas são pseudoaleatórias com semente fixa; para impedir que o compilador elimine buscas cujo resultado não é usado, os retornos são acumulados em uma variável volátil. Avaliaram-se diferentes tamanhos de entrada e um padrão de acesso com localidade controlada, sempre comparando estruturas na mesma tarefa.

B. Trie × Patricia (strings)

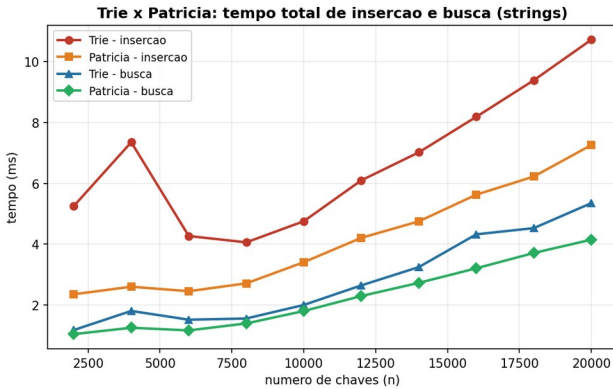


Fig. 7. Tempo total de inserção e busca de n strings aleatórias (comprimento 8).

TABELA II
STRINGS: TEMPO TOTAL EM MS (COMPRIMENTO 8)

n	Trie ins.	Trie bus.	Pat. ins.	Pat. bus.
2.000	5,25	1,18	2,36	1,05
8.000	4,06	1,56	2,71	1,40
14.000	7,01	3,25	4,75	2,73
20.000	10,71	5,35	7,26	4,15

A Patricia é consistentemente mais rápida que a Trie na inserção e na busca e cresce mais suavemente com n. Ambas são $O(m)$, mas a compactação reduz o número de nós visitados e de alocações, melhorando também a localidade de cache, um ganho de fatores

constantes que a análise assintótica, sozinha, não revela.

C. Splay × Treap × BST balanceada (busca aleatória)

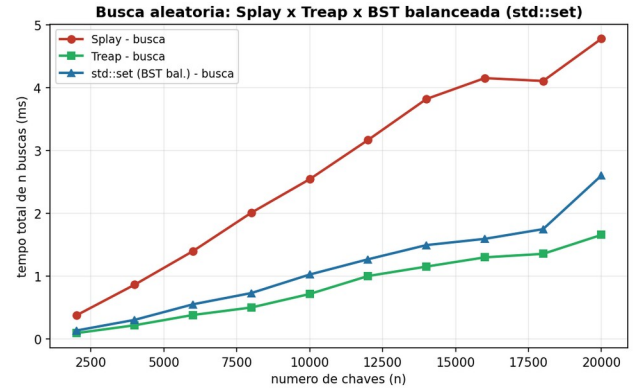


Fig. 8. Tempo total de n buscas aleatórias de inteiros (Splay, Treap e std::set).

TABELA III
INTEIROS: TEMPO TOTAL DE BUSCA EM MS (ACESSO UNIFORME)

n	Splay	Treap	std::set
4.000	0,86	0,21	0,30
10.000	2,54	0,71	1,02
16.000	4,15	1,29	1,59
20.000	4,78	1,65	2,60

Sob acesso uniforme (sem localidade), a Splay é a mais lenta: cada busca realiza rotações (escritas em memória) mesmo sem ganho de localidade. A Treap e o std::set (BST rubro-negra) ficam próximos, coerentes com o custo $O(\log n)$ de ambos, caso em que o comportamento assintótico esperado se confirma na prática.

D. Efeito da localidade de acesso

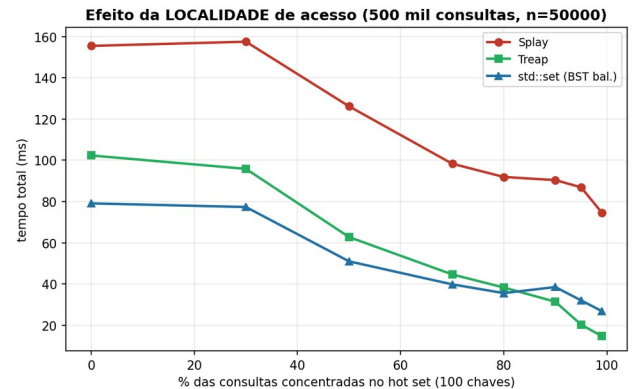


Fig. 9. 500 mil consultas em estrutura com n = 50000, variando a concentração no hot set (100 chaves).

TABELA IV
LOCALIDADE: TEMPO TOTAL EM MS POR CONCENTRAÇÃO P

p	Splay	Treap	std::set
0,00	155,5	102,3	79,1

p	Splay	Treap	std::set
0,50	126,2	62,8	51,0
0,90	90,4	31,4	38,5
0,99	74,7	14,7	26,9

Concentrando progressivamente as consultas em 100 chaves, todas as estruturas melhoram (efeito de cache), mas a Splay é a que mais se beneficia em termos proporcionais: seu tempo cai de ≈ 155 ms (uniforme) para ≈ 75 ms (99% concentrado), redução de $\approx 52\%$, pois as chaves quentes são levadas para perto da raiz e passam a ser encontradas em poucos passos. Ainda assim, em tempo absoluto ela permanece mais lenta que a BST balanceada: o custo constante das rotações não é compensado apenas pela menor profundidade. Trata-se de uma **divergência instrutiva** entre a análise assintótica (que favorece a Splay sob localidade) e o desempenho real (dominado por fatores constantes).

E. KD-Tree $\times$ busca linear (vizinho mais próximo)

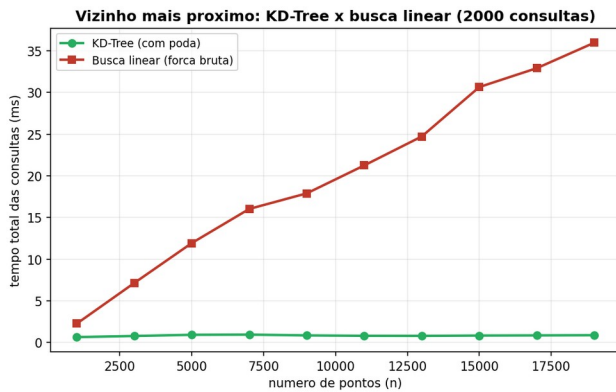


Fig. 10. Tempo de 2000 consultas de vizinho mais próximo: KD-Tree (com poda) $\times$ busca linear.

TABELA V
VIZINHO MAIS PRÓXIMO: TEMPO TOTAL EM MS

n	KD-Tree	Linear	Ganho
1.000	0,65	2,26	3,5×
7.000	0,95	16,02	17×
13.000	0,80	24,72	31×
19.000	0,87	35,97	41×

A busca linear cresce linearmente com n (cada consulta examina todos os pontos), enquanto a KD-Tree permanece praticamente constante graças à poda de regiões. Para $n = 19000$ ela é cerca de **41× mais rápida**. Este é o caso de maior convergência entre teoria e prática: a vantagem $O(\log n)$ médio contra $O(n)$ aparece de forma nítida e cresce com n .

VII. APLICAÇÕES, ANÁLISE CRÍTICA E DISCUSSÃO

Cada estrutura tem um nicho em que suas propriedades estruturais a tornam a escolha adequada:

- **Trie:** autocompletar, dicionários, corretores ortográficos e busca por prefixos, quando a chave é uma string e o compartilhamento de prefixos é frequente.
- **Patricia:** indexação compacta, tabelas de roteamento IP e índices por prefixo, onde a economia de memória da compactação é decisiva.
- **Splay:** caches, montadores e sistemas com acessos altamente desiguais ou localidade temporal, em que itens recentes tendem a ser reaccessados.
- **Treap:** estruturas de busca de propósito geral com balanceamento e implementação simples, e como base de treaps implícitas para operações de sequência.
- **KD-Tree:** busca espacial, sistemas GIS, computação gráfica, reconhecimento de padrões e algoritmos baseados em vizinhos (k-NN).

TABELA VI
SÍNTESE: CENÁRIO IDEAL DE CADA ESTRUTURA

Estrutura	Cenário ideal	Ponto forte
Trie	strings / prefixos	busca $O(m)$
Patricia	índices compactos	memória $O(N)$
Splay	localidade temporal	autoajuste
Treap	uso geral	simplicidade
KD-Tree	busca espacial	poda $O(\log n)$

A. Vantagens, limitações e dificuldades de implementação

A **Trie** é simples, mas pode consumir muita memória; a **Patricia** corrige isso ao custo de split e merge, as partes mais trabalhosas da implementação. A **Splay** tem código de rotação relativamente simples, mas depende de ponteiros de pai e sua vantagem só aparece sob localidade. A **Treap** foi a de implementação mais direta (recursão com rotações), com a ressalva de depender de um bom gerador de aleatoriedade. A remoção da **KD-Tree** (via findMin alternando dimensões) foi a operação mais sutil de todo o trabalho, por exigir cuidado com os casos de borda.

Teoria $\times$ experimento. O caso mais marcante foi a Splay: teoricamente favorecida sob localidade, permaneceu mais lenta em tempo absoluto por causa dos fatores constantes das rotações, embora a tendência prevista (melhora com a localidade) tenha se confirmado. Nos demais casos, teoria e experimento convergiram bem, com destaque para a

KD-Tree contra a busca linear e para a Trie contra a Patricia.

Melhorias possíveis. usar vetores de 26 posições na Trie (em vez de mapa) para acelerar o acesso; construir a KD-Tree de forma balanceada pela mediana; implementar a Splay em versão top-down para reduzir constantes; e adicionar treaps implícitas para operações de sequência.

VIII. CONCLUSÃO

Este trabalho respondeu, de forma fundamentada, às questões norteadoras. **Diferenças:** as estruturas variam no tipo de chave (string, escalar, ponto) e na estratégia de organização (compartilhamento de prefixos, autoajuste por acesso, aleatoriedade, particionamento espacial). **Melhor desempenho:** Patricia para strings; BST balanceada/Treap para busca aleatória de inteiros; Splay sob forte localidade; KD-Tree para busca espacial. **Coerência com a complexidade:** em geral confirmada, sobretudo KD-Tree contra busca linear e Trie contra Patricia, com a exceção instrutiva da Splay. **Maior dificuldade:** a recompactação (merge) da Patricia e a remoção da KD-Tree.

Em síntese, o estudo evidenciou a relação direta entre **propriedades estruturais**, **complexidade algorítmica** e **desempenho observado**. Compreender essas relações é o que permite justificar tecnicamente a escolha de uma estrutura para um problema computacional específico, objetivo central deste trabalho. Não há vencedora universal: cada estrutura domina o nicho para o qual foi projetada, e a decisão correta depende do padrão de dados e de acesso da aplicação.

REFERÊNCIAS

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest e C. Stein, Algoritmos: Teoria e Prática, 3. ed. Rio de Janeiro: Elsevier, 2012.
- [2] R. Sedgwick e K. Wayne, Algorithms, 4. ed. Boston: Addison-Wesley, 2011.
- [3] D. D. Sleator e R. E. Tarjan, "Self-adjusting binary search trees," Journal of the ACM, v. 32, n. 3, p. 652-686, 1985.
- [4] C. R. Aragon e R. Seidel, "Randomized search trees," in Proc. 30th FOCS, 1989, p. 540-545.
- [5] J. L. Bentley, "Multidimensional binary search trees used for associative searching," Communications of the ACM, v. 18, n. 9, p. 509-517, 1975.
- [6] D. R. Morrison, "PATRICIA: practical algorithm to retrieve information coded in alphanumeric," Journal of the ACM, v. 15, n. 4, p. 514-534, 1968.
- [7] Repositório do projeto. Código-fonte completo em C++17. Disponível em: <https://github.com/mariaeduarda-ab/trabalho-arvores-avancadas>.